

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ
Факультет компьютерных систем и сетей
Кафедра электронных вычислительных машин

Лабораторная работа №1

Разработка серверной части прикладной программы

Студент:
Преподаватель:

А.В. Губаревич
С.С. Силич

МИНСК 2026

СОДЕРЖАНИЕ

Введение.....	Ошибка! Закладка не определена.
1 Технические требования.....	Ошибка! Закладка не определена.
1.1 Описание реляционной модели.....	Ошибка! Закладка не определена.
1.2 Описание таблиц.....	Ошибка! Закладка не определена.
1.3 Выделение справочных и основных таблиц	6
1.4 Выделение прав доступа	Ошибка! Закладка не определена.
1.5 Определение требований к серверной части	7
2 Программирование серверной части.....	Ошибка! Закладка не определена.
2.1 Создание скриптов.....	10
2.2 Реализация HTTP-сервера.....	Ошибка! Закладка не определена.
Заключение	Ошибка! Закладка не определена.
Приложение А - Листинг кода.....	Ошибка! Закладка не определена.

ВВЕДЕНИЕ

В современной разработке программного обеспечения важную роль играет разделение приложения на клиентскую и серверную части, что позволяет обеспечить безопасность, масштабируемость и удобство поддержки системы. Серверная часть (backend) отвечает за обработку бизнес-логики, взаимодействие с базой данных и обеспечение доступа к данным через стандартизированные интерфейсы.

Целью данной лабораторной работы является разработка серверной части прикладной программы для управления базой данных университета, созданной в ходе предыдущих работ.

Работа основана на уже существующей модели данных, которая включает таблицы студентов, сотрудников, учебных групп, оценок, расписания и справочников. В ходе выполнения были разработаны спецификации, реализованы скрипты для создания и наполнения базы данных, а также обеспечена поддержка основных CRUD-операций через HTTP-запросы.

Разработанная серверная часть позволяет организовать удобный и безопасный доступ к данным, а также служит основой для дальнейшего развития приложения — например, создания веб-интерфейса или мобильного клиента.

1 РАЗРАБОТКА СПЕЦИФИКАЦИЙ СЕРВЕРНОЙ ЧАСТИ

1.1 Описание реляционной модели

В данном случае внесение изменений не требуется и реляционная схема остается такого же вида, как и в предыдущих работах.

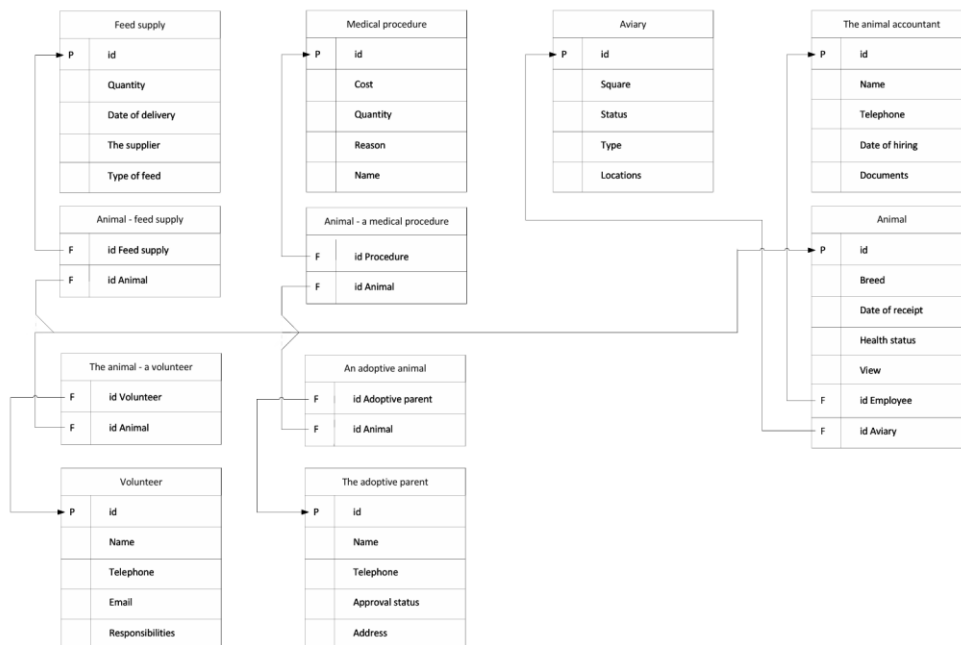


Рисунок 1.1 – UML-диаграмма

1.2 Описание таблиц

Таблица `Aviary` представляет данные о вольерах для содержания животных.

Описание полей таблицы `Aviary`:

`id`: идентификатор вольера. Первичный ключ (автоинкремент);

`square`: площадь вольера (в m^2);

`status`: статус вольера (Active, Maintenance, Cleaning, Quarantine, Renovation);

`type`: тип вольера (Outdoor, Indoor, Mixed, Isolation);

`location`: местоположение вольера на территории приюта.

Таблица `Adoptive_parent` представляет данные о потенциальных усыновителях животных.

Описание полей таблицы `Adoptive_parent`:

id: идентификатор усыновителя. Первичный ключ (автоинкремент);
name: ФИО усыновителя;
telephone: контактный телефон;
approval_status: статус одобрения заявки (Approved, Pending, Rejected);
address: адрес проживания.

Таблица `Employee` представляет данные о сотрудниках приюта.

Описание полей таблицы `Employee`:

id: идентификатор сотрудника. Первичный ключ (автоинкремент);
name: ФИО сотрудника;
telephone: контактный телефон;
hire_date: дата приема на работу;
post: должность (Ветеринар, Зоотехник, Смотритель, Администратор, Волонтер).

Таблица `Medical_procedure` представляет данные о медицинских процедурах.

Описание полей таблицы `Medical_procedure`:

id: идентификатор процедуры. Первичный ключ (автоинкремент);
cost: стоимость процедуры;
amount: количество процедур;
reason: причина проведения;
name: название процедуры.

Таблица `Feed_supply` представляет данные о поставках корма.

Описание полей таблицы `Feed_supply`:

id: идентификатор поставки. Первичный ключ (автоинкремент);
quantity: количество корма;
delivery_date: дата поставки;
the_supplier: поставщик корма;
type_of_feed: тип корма.

Таблица `Animal` представляет данные о животных в приюте. Это центральная сущность системы.

Описание полей таблицы `Animal`:

id: идентификатор животного. Первичный ключ (автоинкремент);
breed: порода животного;
date_of_receipt: дата поступления в приют;
state_of_health: состояние здоровья (Здоров, Лечение, Карантин);
type: тип животного (Кошка, Собака, Птица, Грызун);
id_aviary: внешний ключ на таблицу `Aviary`, указывает на вольер, где содержится животное;

`id_employee`: внешний ключ на таблицу `Employee`, указывает на ответственного сотрудника.

Таблица `Volunteer` представляет данные о волонтерах приюта.

Описание полей таблицы `Volunteer`:

`id`: идентификатор волонтера. Первичный ключ (автоинкремент);

`name`: ФИО волонтера;

`telephone`: контактный телефон;

`email`: адрес электронной почты;

`responsibilities`: обязанности волонтера.

Связующие таблицы (для отношений "многие-ко-многим"):

`Adoptive_animal`: связывает усыновителей и животных (животное может быть усыновлено, у усыновителя может быть несколько животных). Составной уникальный ключ (`id_adoptive`, `id_animal`).

`Animal_feed_supply`: связывает животных и поставки корма (животное потребляет разные корма, поставка корма может использоваться для многих животных). Составной уникальный ключ (`id_animal`, `id_feed_supply`).

`Animal_volunteer`: связывает животных и волонтеров (за животным могут ухаживать несколько волонтеров, волонтер может ухаживать за несколькими животными). Составной уникальный ключ (`id_volunteer`, `id_animal`).

`Animal_medical_procedure`: связывает животных и медицинские процедуры (животное может проходить множество процедур, процедура может проводиться многим животным). Составной уникальный ключ (`id_animal`, `id_medical`).

1.3 Выделение справочных и основных таблиц

В представленной схеме базы данных «Приют для животных» к категории справочных таблиц относятся таблицы, содержащие predetermined, классифицирующие данные. Это таблицы, которые определяют статусы, типы и другие атрибуты, используемые для заполнения выпадающих списков и обеспечения целостности данных. К ним можно отнести `Medical_procedure`, `Feed_supply`, а также таблицы, играющие роль классификаторов (например, перечень возможных статусов вольера или видов животных, хотя эти перечни реализованы как поля с ограничениями, в полноценной схеме они могли бы быть вынесены в отдельные справочники).

Основными таблицами, содержащими ключевые бизнес-сущности и операционные данные, являются `Animal`, `Volunteer`, `Employee`, `Adoptive_parent` и `Aviary`. Именно они хранят информацию о питомцах, сотрудниках, волонтерах и усыновителях.

Центральной и самой динамичной таблицей всей системы является `Animal`, так как она аккумулирует информацию о каждом питомце и связана со всеми остальными ключевыми сущностями. Работа с карточкой животного является основной функцией приложения.

1.4 Выделение прав доступа

Роль «Пользователь» предоставляет авторизованному субъекту полный спектр прав на работу с основными операционными данными. Это включает неограниченный просмотр (`SELECT`), модификацию (`INSERT`, `UPDATE`) и сохранение результатов запросов для всех таблиц, содержащих ключевые сущности (животные, сотрудники, волонтеры, усыновители, вольеры). Однако права на изменение справочных таблиц заблокированы, что предотвращает случайное или несанкционированное изменение системных данных.

1.5 Определение требований к серверной части

Серверная часть прикладной программы реализована в виде HTTP-сервера на основе фреймворка FastAPI. Тела ответов сервера, так же, как и тела запросов, представлены в формате JSON.

Для взаимодействия с ресурсами (таблицами) используются стандартные HTTP-методы:

`GET` – получение данных о ресурсе;

`POST` – создание нового ресурса;

`PUT` – обновление существующего ресурса;

`DELETE` – удаление ресурса.

Каждый ресурс доступен по уникальному URL, который сопоставляется с соответствующей таблицей в базе данных через механизм `TABLE_MAPPING`:

`/api/animals` – работа с таблицей `Animal` (животные);

`/api/aviaries` – работа с таблицей `Aviary` (вольеры);

`/api/employees` – работа с таблицей `Employee` (сотрудники);

`/api/volunteers` – работа с таблицей `Volunteer` (волонтеры);

`/api/adoptive-parents` – работа с таблицей `Adoptive_parent` (усыновители);

`/api/medical-procedures` – работа с таблицей `Medical_procedure` (медицинские процедуры);

`/api/feed-supplies` – работа с таблицей `Feed_supply` (поставки корма);

`/api/adoptive-animals` – работа со связующей таблицей `Adoptive_animal`;

`/api/animal-volunteers` – работа со связующей таблицей `Animal_volunteer`;
`/api/animal-feed-supplies` – работа со связующей таблицей `Animal_feed_supply`;
`/api/animal-medical-procedures` – работа со связующей таблицей `Animal_medical_procedure`.

Помимо стандартных CRUD-операций, API предоставляет специальные эндпоинты для получения аналитических отчетов, реализованных в лабораторных работах №5 и №6:

`GET /api/reports/animals_treatment` – животные на лечении или карантине;

`GET /api/reports/employees_recent` – сотрудники, принятые после 2021 года;

`GET /api/reports/animals_active_aviaries` – животные в активных вольерах;

`GET /api/reports/volunteers_search` – волонтеры по уходу за собаками и кошками;

`GET /api/reports/medical_procedures_cost_range` – медицинские процедуры стоимостью от 2000 до 3500 рублей;

`GET /api/reports/animals_with_volunteers` – животные с закрепленными волонтерами;

`GET /api/reports/animals_count_by_type` – статистика животных по типам;

`GET /api/reports/unique_breeds_active_aviaries` – уникальные породы в активных вольерах;

`GET /api/reports/max_outdoor_square` – вольеры типа Outdoor с максимальной площадью;

`GET /api/reports/total_feed_supply` – общее количество поставленного корма;

`GET /api/reports/aviary_avg_square` – типы вольеров со средней площадью более 20 м²;

`GET /api/reports/animals_both_cheap_expensive` – животные, прошедшие и дешевые, и дорогие процедуры;

`GET /api/reports/empty_aviaries` – пустые вольеры.

Для обеспечения сохранности данных реализован функционал резервного копирования:

`GET /api/backup/create` – создать резервную копию всей базы данных;

`GET /api/backup/create?table=animals` – создать резервную копию конкретной таблицы;

GET /api/backup/list — получить список доступных резервных копий;

GET /api/backup/download/{filename} — скачать файл резервной копии.

Сервер настроен с использованием middleware CORS для корректного взаимодействия с клиентскими веб-приложениями, размещенными на других доменах.

Серверная часть прикладной программы предоставляет следующие операции для работы с базой данных:

- просмотр таблиц с поддержкой пагинации и фильтрации по ID;
- добавление записей в таблицы;
- обновление записей в таблицах;
- удаление записей из таблиц;
- выполнение специальных аналитических запросов;
- создание резервных копий базы данных;
- скачивание файлов резервных копий;
- экспорт результатов запросов в JSON формат.

2 ПРОГРАММИРОВАНИЕ СЕРВЕРНОЙ ЧАСТИ

2.1 Создание скриптов

Для создания таблиц в базе данных используется следующий скрипт:

```
-- Создание таблицы Aviary (Вольеры)
CREATE TABLE IF NOT EXISTS public."Aviary" (
    id bigint NOT NULL GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
    square numeric NOT NULL,
    status character varying(15) NOT NULL,
    type character varying(30) NOT NULL,
    location character varying(30) NOT NULL
);

-- Создание таблицы Adoptive_parent (Усыновители)
CREATE TABLE IF NOT EXISTS public."Adoptive_parent" (
    id bigint NOT NULL GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
    snp character varying(50) NOT NULL,
    telephone character varying(20) NOT NULL,
    approval_status character varying(15) NOT NULL,
    address character varying(50) NOT NULL
);

-- Создание таблицы Employee (Сотрудники)
CREATE TABLE IF NOT EXISTS public."Employee" (
    id bigint NOT NULL GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
    snp character varying(50) NOT NULL,
    telephone character varying(20) NOT NULL,
    hire_date date NOT NULL,
    post character varying(30) NOT NULL
);

-- Создание таблицы Medical_procedure (Медицинские
процедуры)
CREATE TABLE IF NOT EXISTS public."Medical_procedure" (
    id bigint NOT NULL GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
    cost integer NOT NULL,
    amount integer NOT NULL,
    reason character varying(50) NOT NULL,
    name character varying(30) NOT NULL
);
```

```

-- Создание таблицы Feed_supply (Поставки корма)
CREATE TABLE IF NOT EXISTS public."Feed_supply" (
    id bigint NOT NULL GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
    quantity integer NOT NULL,
    delivery_date date NOT NULL,
    the_supplier character varying(30) NOT NULL,
    type_of_feed character varying(30) NOT NULL
);

-- Создание таблицы Animal (Животные)
CREATE TABLE IF NOT EXISTS public."Animal" (
    id bigint NOT NULL GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
    breed character varying(30) NOT NULL,
    date_of_receipt date NOT NULL,
    state_of_health character varying(30) NOT NULL,
    type character varying(30) NOT NULL,
    id_avinary bigint NOT NULL,
    id_employee bigint NOT NULL,
    CONSTRAINT fk_animal_avinary FOREIGN KEY (id_avinary)
REFERENCES public."Aviary"(id),
    CONSTRAINT fk_animal_employee FOREIGN KEY (id_employee)
REFERENCES public."Employee"(id)
);

-- Создание таблицы Volunteer (Волонтеры)
CREATE TABLE IF NOT EXISTS public."Volunteer" (
    id bigint NOT NULL GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
    snp character varying(50) NOT NULL,
    telephone character varying(20) NOT NULL,
    email character varying(100) NOT NULL,
    duty character varying(50) NOT NULL
);

-- Создание связующей таблицы Adoptive_animal
CREATE TABLE IF NOT EXISTS public."Adoptive_animal" (
    id bigint NOT NULL GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
    id_adoptive bigint NOT NULL REFERENCES
public."Adoptive_parent"(id),
    id_animal bigint NOT NULL REFERENCES
public."Animal"(id),
    UNIQUE(id_adoptive, id_animal)
);

```

```

-- Создание связующей таблицы Animal_feed_supply
CREATE TABLE IF NOT EXISTS public."Animal_feed_supply" (
    id bigint NOT NULL GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
    id_animal bigint NOT NULL REFERENCES
public."Animal"(id),
    id_feed_supply bigint NOT NULL REFERENCES
public."Feed_supply"(id),
    UNIQUE(id_animal, id_feed_supply)
);

-- Создание связующей таблицы Animal_volunteer
CREATE TABLE IF NOT EXISTS public."Animal_volunteer" (
    id bigint NOT NULL GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
    id_volunteer bigint NOT NULL REFERENCES
public."Volunteer"(id),
    id_animal bigint NOT NULL REFERENCES
public."Animal"(id),
    UNIQUE(id_volunteer, id_animal)
);

-- Создание связующей таблицы Animal_medical_procedure
CREATE TABLE IF NOT EXISTS
public."Animal_medical_procedure" (
    id bigint NOT NULL GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
    id_animal bigint NOT NULL REFERENCES
public."Animal"(id),
    id_medical bigint NOT NULL REFERENCES
public."Medical_procedure"(id),
    procedure_date date,
    UNIQUE(id_animal, id_medical)
);

```

2.2 Реализация HTTP-сервера

Для создания серверной части был использован подход Code-first и фреймворк FastAPI. Взаимодействие с базой данных PostgreSQL осуществляется через библиотеку psycopg2 (клиентскую библиотеку на языке C, которая является надстройкой над libpq). Для обеспечения гибкости и избежания жесткой привязки к ORM, прямой доступ к данным организован через слой абстракции – класс DatabaseManager в модуле database.py, предоставляющий методы execute_query, get_table_data, insert_record, update_record и delete_record.

Использование библиотеки `libpq` реализовано опосредованно через драйвер `psycopg2`, что является современным и эффективным подходом. Это позволяет абстрагироваться от низкоуровневых деталей работы с PostgreSQL, но при этом гарантирует надежное и безопасное выполнение всех SQL-запросов, обеспечивая стабильное соединение с базой данных и корректную обработку результатов.

Для каждой сущности автоматически генерируются стандартные CRUD-эндпоинты через параметризованные обработчики в файле `main.py`. Это устраняет необходимость создания отдельных контроллеров для каждой таблицы, делая код более лаконичным и поддерживаемым. Валидация входящих данных осуществляется с помощью Pydantic-моделей, определенных в файле `models.py`.

Для тестирования GET-запросов, благодаря встроенной в FastAPI поддержке Swagger UI, использовалась интерактивная документация API, доступная по маршруту `/docs`. На рисунке 3.1 отображен интерфейс Swagger с полным списком доступных эндпоинтов, а на рисунке 3.2 – результат выполнения GET-запроса к таблице `Animal`, полученный через браузер.

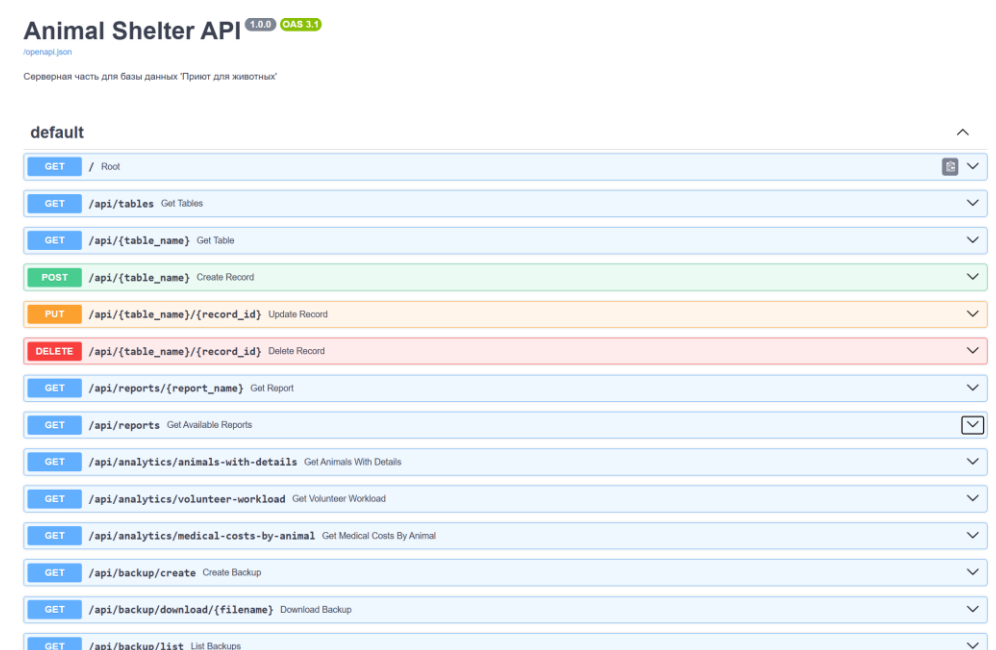


Рисунок 2.1 – Интерактивная документация API (Swagger UI)



Curl

```
curl -X 'GET' \
'http://localhost:8000/api/animals?page=1&limit=100' \
-H 'accept: application/json'
```

Request URL

```
http://localhost:8000/api/animals?page=1&limit=100
```

Server response

Code	Details
200	Response body <pre>{ "data": [{ "id": 1, "breed": "Британская короткошерстная", "date_of_receipt": "2024-01-15", "state_of_health": "Здоров", "type": "Кошка", "id_avary": 1, "id_employee": 1 }, { "id": 2, "breed": "Немецкая овчарка", "date_of_receipt": "2024-01-16", "state_of_health": "Здоров", "type": "Собака", "id_avary": 2, "id_employee": 2 }, { "id": 3, "breed": "Сиамская", "date_of_receipt": "2024-01-17", "state_of_health": "Лечение", "type": "Кошка", "id_avary": 3, </pre> Response headers

Рисунок 2.2 – Результаты выполнения GET-запроса к эндпоинту /api/animals

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы была успешно разработана серверная часть прикладной программы для управления базой данных университета. Была уточнена реляционная схема, определены роли пользователей и суперпользователей, а также реализован HTTP-сервер, предоставляющий RESTful API для работы с данными.

Разработанная серверная часть является гибким и масштабируемым решением, которое может быть интегрировано с различными клиентскими приложениями. Она соответствует современным стандартам разработки backend-систем и может служить основой для дальнейшего расширения функционала, такого как добавление аутентификации, логирования или интеграции с внешними сервисами.

Таким образом, поставленные цели и задачи лабораторной работы выполнены в полном объеме.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг кода

```
from fastapi import FastAPI, HTTPException, Query
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import FileResponse,
JSONResponse
from contextlib import asynccontextmanager
from typing import Optional, Dict, Any, List
import os

from database import db_manager
from models import *
from queries import ShelterQueries
from backup import backup_manager

# Lifespan контекстный менеджер для управления ресурсами
@asynccontextmanager
async def lifespan(app: FastAPI):
    # Код при запуске
    print("Сервер 'Приют для животных' запускается...")
    yield
    # Код при остановке
    print("Сервер останавливается, закрываем
соединения...")
    db_manager.close_connection()

# Создаем приложение FastAPI с lifespan
app = FastAPI(
    title="Animal Shelter API",
    description="Серверная часть для базы данных 'Приют
для животных'",
    version="1.0.0",
    lifespan=lifespan
)

# Настройка CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Словарь для соответствия URL и имен таблиц в БД
TABLE_MAPPING = {
    "animals": "Animal",
```



```

        "aviaries": "Aviary",
        "employees": "Employee",
        "volunteers": "Volunteer",
        "adoptive-parents": "Adoptive_parent",
        "medical-procedures": "Medical_procedure",
        "feed-supplies": "Feed_supply",
        "adoptive-animals": "Adoptive_animal",
        "animal-volunteers": "Animal_volunteer",
        "animal-feed-supplies": "Animal_feed_supply",
        "animal-medical-procedures":
"Animal_medical_procedure"
    }

@app.get("/")
async def root():
    """Корневой эндпоинт"""
    return {
        "message": "Animal Shelter Database API",
        "version": "1.0.0",
        "tables": list(TABLE_MAPPING.keys())
    }

@app.get("/api/tables")
async def get_tables():
    """Получить список всех таблиц в базе данных"""
    try:
        tables = db_manager.get_table_names()
        return {"tables": tables}
    except Exception as e:
        raise HTTPException(status_code=500,
detail=str(e))

# ===== УНИВЕРСАЛЬНЫЕ CRUD ЭНДПОИНТЫ =====

@app.get("/api/{table_name}")
async def get_table(
    table_name: str,
    id: Optional[int] = None,
    page: int = Query(1, ge=1),
    limit: int = Query(100, ge=1, le=1000)
):
    """
    Получить данные из таблицы с поддержкой фильтрации
по ID и пагинации
    """
    try:
        db_table_name = TABLE_MAPPING.get(table_name)
        if not db_table_name:

```

```

        raise HTTPException(status_code=404,
detail="Table not found")

    filters = {}
    if id:
        filters["id"] = id

    offset = (page - 1) * limit
    data = db_manager.get_table_data(db_table_name,
filters, limit, offset)

    return {
        "data": data,
        "page": page,
        "limit": limit,
        "count": len(data)
    }
except Exception as e:
    raise HTTPException(status_code=500,
detail=str(e))

@app.post("/api/{table_name}")
async def create_record(table_name: str, data: Dict[Any,
Any]):
    """
    Создать новую запись в таблице
    """
    try:
        db_table_name = TABLE_MAPPING.get(table_name)
        if not db_table_name:
            raise HTTPException(status_code=404,
detail="Table not found")

        result =
db_manager.insert_record(db_table_name, data)
        return {"message": "Record created
successfully", "data": result}
    except Exception as e:
        raise HTTPException(status_code=500,
detail=str(e))

@app.put("/api/{table_name}/{record_id}")
async def update_record(table_name: str, record_id: int,
data: Dict[Any, Any]):
    """
    Обновить запись в таблице по ID
    """
    try:
        db_table_name = TABLE_MAPPING.get(table_name)
        if not db_table_name:

```

```

        raise HTTPException(status_code=404,
detail="Table not found")

        result =
db_manager.update_record(db_table_name, record_id, data)
        return {"message": "Record updated
successfully", "data": result}
    except Exception as e:
        raise HTTPException(status_code=500,
detail=str(e))

@app.delete("/api/{table_name}/{record_id}")
async def delete_record(table_name: str, record_id:
int):
    """
    Удалить запись из таблицы по ID
    """
    try:
        db_table_name = TABLE_MAPPING.get(table_name)
        if not db_table_name:
            raise HTTPException(status_code=404,
detail="Table not found")

        result =
db_manager.delete_record(db_table_name, record_id)
        return {"message": "Record deleted
successfully", "data": result}
    except Exception as e:
        raise HTTPException(status_code=500,
detail=str(e))

# ===== СПЕЦИАЛЬНЫЕ ЗАПРОСЫ (ИЗ ЛАБОРАТОРНЫХ РАБОТ)
=====

@app.get("/api/reports/{report_name}")
async def get_report(report_name: str):
    """
    Выполнить predetermined отчет из лабораторных
работ
    """
    try:
        query = ShelterQueries.QUERIES.get(report_name)
        if not query:
            raise HTTPException(status_code=404,
detail="Report not found")

        result = db_manager.execute_query(query)
        description =
ShelterQueries.QUERY_DESCRIPTIONS.get(report_name,
report_name)

```

```

        return {
            "report_name": description,
            "data": result
        }
    except Exception as e:
        raise HTTPException(status_code=500,
detail=str(e))

@app.get("/api/reports")
async def get_available_reports():
    """Получить список доступных отчетов"""
    try:
        reports = [
            {"id": name, "description": desc}
            for name, desc in
ShelterQueries.QUERY_DESCRIPTIONS.items()
            if name in ShelterQueries.QUERIES
        ]
        return {"reports": reports}
    except Exception as e:
        raise HTTPException(status_code=500,
detail=str(e))

# ===== АНАЛИТИЧЕСКИЕ ЗАПРОСЫ =====

@app.get("/api/analytics/animals-with-details")
async def get_animals_with_details():
    """
        Получить информацию о животных с названиями вольеров
        и именами сотрудников
    """
    try:
        result =
db_manager.execute_query(ShelterQueries.ANIMALS_WITH_DETAIL
S)
        return {"data": result}
    except Exception as e:
        raise HTTPException(status_code=500,
detail=str(e))

@app.get("/api/analytics/volunteer-workload")
async def get_volunteer_workload():
    """
        Получить нагрузку на волонтеров (сколько животных за
        каждым)
    """
    try:

```

```

        result
db_manager.execute_query(ShelterQueries.VOLUNTEER_WORKLOAD)
        return {"data": result}
    except Exception as e:
        raise HTTPException(status_code=500,
detail=str(e))

@app.get("/api/analytics/medical-costs-by-animal")
async def get_medical_costs_by_animal():
    """
    Получить суммарные медицинские расходы по каждому
    животному
    """
    try:
        result
db_manager.execute_query(ShelterQueries.MEDICAL_COSTS_BY_AN
IMAL)
        return {"data": result}
    except Exception as e:
        raise HTTPException(status_code=500,
detail=str(e))

# ===== РЕЗЕРВНОЕ КОПИРОВАНИЕ =====

@app.get("/api/backup/create")
async def create_backup(table: Optional[str] = None):
    """
    Создать резервную копию базы данных или конкретной
    таблицы
    """
    try:
        if table:
            db_table = TABLE_MAPPING.get(table)
            if not db_table:
                raise HTTPException(status_code=404,
detail="Table not found")
            filepath
backup_manager.create_backup(db_table)
            message = f"Backup of table '{db_table}'
created successfully"
        else:
            filepath = backup_manager.create_backup()
            message = "Full database backup created
successfully"

        filename = os.path.basename(filepath)
        return {
            "message": message,
            "filename": filename,
            "path": filepath

```

```

    }
    except Exception as e:
        raise HTTPException(status_code=500,
detail=str(e))

@app.get("/api/backup/download/{filename}")
async def download_backup(filename: str):
    """
    Скачать файл резервной копии
    """
    filepath = os.path.join("exports", filename)
    if not os.path.exists(filepath):
        raise HTTPException(status_code=404,
detail="Backup file not found")

    return FileResponse(
        filepath,
        media_type="application/octet-stream",
        filename=filename
    )

@app.get("/api/backup/list")
async def list_backups():
    """
    Получить список доступных резервных копий
    """
    try:
        backups = []
        if os.path.exists("exports"):
            for file in os.listdir("exports"):
                if file.endswith(".sql"):
                    filepath = os.path.join("exports",
file)

                    size = os.path.getsize(filepath)
                    modified =
os.path.getmtime(filepath)
                    backups.append({
                        "filename": file,
                        "size": size,
                        "modified": modified
                    })
            return {"backups": backups}
    except Exception as e:
        raise HTTPException(status_code=500,
detail=str(e))

# ===== ЭКСПОРТ В JSON =====

@app.get("/api/export/{table_name}")

```

```

async def export_table_to_json(table_name: str):
    """
    Экспортировать данные таблицы в JSON формате
    """
    try:
        db_table_name = TABLE_MAPPING.get(table_name)
        if not db_table_name:
            raise HTTPException(status_code=404,
                                detail="Table not found")

        data = db_manager.get_table_data(db_table_name,
                                          limit=10000)

        return JSONResponse(
            content={
                "table": db_table_name,
                "count": len(data),
                "data": data
            }
        )
    except Exception as e:
        raise HTTPException(status_code=500,
                                detail=str(e))

if __name__ == "__main__":
    import uvicorn

    print("=" * 50)
    print("Сервер 'Приют для животных' запускается...")
    print("Документация API будет доступна по адресу: http://localhost:8000/docs")
    print("=" * 50)
    uvicorn.run(app, host="0.0.0.0", port=8000)

    from pydantic import BaseModel, Field, validator
    from typing import Optional, List
    from datetime import date, time

    # ===== МОДЕЛИ ДЛЯ ТАБЛИЦ =====

    class AnimalBase(BaseModel):
        breed: str
        date_of_receipt: date
        state_of_health: str
        type: str
        id_avairy: int
        id_employee: int

    class AnimalCreate(AnimalBase):
        pass

```

```

class AnimalUpdate(BaseModel):
    breed: Optional[str] = None
    date_of_receipt: Optional[date] = None
    state_of_health: Optional[str] = None
    type: Optional[str] = None
    id_aviary: Optional[int] = None
    id_employee: Optional[int] = None

class Animal(AnimalBase):
    id: int

class AviaryBase(BaseModel):
    square: float
    status: str
    type: str
    location: str

class AviaryCreate(AviaryBase):
    pass

class AviaryUpdate(BaseModel):
    square: Optional[float] = None
    status: Optional[str] = None
    type: Optional[str] = None
    location: Optional[str] = None

class Aviary(AviaryBase):
    id: int

class EmployeeBase(BaseModel):
    snp: str
    telephone: str
    hire_date: date
    post: str

class EmployeeCreate(EmployeeBase):
    pass

class EmployeeUpdate(BaseModel):
    snp: Optional[str] = None
    telephone: Optional[str] = None
    hire_date: Optional[date] = None
    post: Optional[str] = None

class Employee(EmployeeBase):
    id: int

class VolunteerBase(BaseModel):
    snp: str
    telephone: str
    email: str
    duty: str

```



```

class VolunteerCreate(VolunteerBase):
    pass

class VolunteerUpdate(BaseModel):
    snp: Optional[str] = None
    telephone: Optional[str] = None
    email: Optional[str] = None
    duty: Optional[str] = None

class Volunteer(VolunteerBase):
    id: int

class AdoptiveParentBase(BaseModel):
    snp: str
    telephone: str
    approval_status: str
    address: str

class AdoptiveParentCreate(AdoptiveParentBase):
    pass

class AdoptiveParentUpdate(BaseModel):
    snp: Optional[str] = None
    telephone: Optional[str] = None
    approval_status: Optional[str] = None
    address: Optional[str] = None

class AdoptiveParent(AdoptiveParentBase):
    id: int

class MedicalProcedureBase(BaseModel):
    cost: int
    amount: int
    reason: str
    name: str

class MedicalProcedureCreate(MedicalProcedureBase):
    pass

class MedicalProcedureUpdate(BaseModel):
    cost: Optional[int] = None
    amount: Optional[int] = None
    reason: Optional[str] = None
    name: Optional[str] = None

class MedicalProcedure(MedicalProcedureBase):
    id: int

class FeedSupplyBase(BaseModel):
    quantity: int
    delivery_date: date

```

```

        the_supplier: str
        type_of_feed: str

class FeedSupplyCreate(FeedSupplyBase):
    pass

class FeedSupplyUpdate(BaseModel):
    quantity: Optional[int] = None
    delivery_date: Optional[date] = None
    the_supplier: Optional[str] = None
    type_of_feed: Optional[str] = None

class FeedSupply(FeedSupplyBase):
    id: int

# ===== СВЯЗУЮЩИЕ ТАБЛИЦЫ =====

class AdoptiveAnimalBase(BaseModel):
    id_adoptive: int
    id_animal: int

class AdoptiveAnimalCreate(AdoptiveAnimalBase):
    pass

class AdoptiveAnimal(AdoptiveAnimalBase):
    id: int

class AnimalVolunteerBase(BaseModel):
    id_volunteer: int
    id_animal: int

class AnimalVolunteerCreate(AnimalVolunteerBase):
    pass

class AnimalVolunteer(AnimalVolunteerBase):
    id: int

class AnimalFeedSupplyBase(BaseModel):
    id_animal: int
    id_feed_supply: int

class AnimalFeedSupplyCreate(AnimalFeedSupplyBase):
    pass

class AnimalFeedSupply(AnimalFeedSupplyBase):
    id: int

class AnimalMedicalProcedureBase(BaseModel):
    id_animal: int
    id_medical: int
    procedure_date: Optional[date] = None

```

```

class
AnimalMedicalProcedureCreate (AnimalMedicalProcedureBase):
    pass

class
AnimalMedicalProcedure (AnimalMedicalProcedureBase):
    id: int

# ===== МОДЕЛИ ДЛЯ ОТВЕТОВ =====

class ResponseModel (BaseModel):
    message: str
    data: Optional[List] = None

class PaginatedResponse (BaseModel):
    data: List
    page: int
    limit: int
total: Optional[int] = None

# SQL ЗАПРОСЫ ИЗ ЛАБОРАТОРНЫХ РАБОТ №5-6

class ShelterQueries:
    """Класс с предопределенными SQL запросами"""

    # JP5: Простые запросы с WHERE
    ANIMALS_TREATMENT = """
        SELECT id AS "ID", type AS "Тип животного",
        breed AS "Порода",
        state_of_health AS "Состояние здоровья",
        date_of_receipt AS "Дата поступления"
        FROM "Animal"
        WHERE state_of_health IN ('Лечение',
        'Карантин')
        ORDER BY type, breed
    """

    EMPLOYEES_RECENT = """
        SELECT snp AS "ФИО сотрудника", post AS
        "Должность",
        telephone AS "Телефон", hire_date AS
        "Дата приема на работу"
        FROM "Employee"
        WHERE hire_date > '2021-01-01'
        ORDER BY hire_date DESC
    """

    ANIMALS_ACTIVE_AVIARIES = """
        SELECT a.type AS "Тип животного", a.breed AS
        "Порода",
        av.square AS "Площадь вольера",
        av.status AS "Статус вольера",

```

```

        av.location AS "Местоположение"
    FROM "Animal" a
    INNER JOIN "Aviary" av ON a.id_aviary = av.id
    WHERE av.status = 'Active'
    ORDER BY a.type, av.square DESC
""

VOLUNTEERS_SEARCH = ""
    SELECT snp AS "ФИО волонтера", telephone AS
"Телефон",
        email AS "Email", duty AS "Обязанности"
    FROM "Volunteer"
    WHERE duty LIKE '%собак%' OR duty LIKE
'%кошками%'
    ORDER BY duty
""

MEDICAL_PROCEDURES_COST_RANGE = ""
    SELECT name AS "Название процедуры", cost AS
"Стоимость, руб",
        amount AS "Количество", reason AS
"Причина проведения"
    FROM "Medical_procedure"
    WHERE cost BETWEEN 2000 AND 3500
    ORDER BY cost ASC
""

# ЛР5: JOIN запросы
ANIMALS_WITH_VOLUNTEERS = ""
    SELECT a.id AS "ID животного", a.type AS "Тип",
a.breed AS "Порода",
        v.snp AS "ФИО волонтера", v.duty AS
"Обязанности волонтера"
    FROM "Animal" a
    JOIN "Animal_volunteer" av ON a.id =
av.id_animal
    JOIN "Volunteer" v ON av.id_volunteer = v.id
    ORDER BY a.type, a.breed
""

# ЛР6: Запросы с агрегатными функциями
ANIMALS_COUNT_BY_TYPE = ""
    SELECT type AS "Тип животного", COUNT(*) AS
"Количество"
    FROM "Animal"
    GROUP BY type
    ORDER BY "Количество" DESC
""

UNIQUE_BREEDS_IN_ACTIVE_AVIARIES = ""
    SELECT COUNT(DISTINCT a.breed) AS "Уникальных
пород"

```

```

        FROM "Animal" a
        JOIN "Aviary" av ON a.id_aviary = av.id
        WHERE av.status = 'Active'
    """

    MAX_OUTDOOR_SQUARE = """
        SELECT type AS "Тип вольера", square AS
"Площадь",
                status AS "Статус", location AS
"Местоположение"
        FROM "Aviary"
        WHERE type = 'Outdoor' AND square = (SELECT
MAX(square) FROM "Aviary" WHERE type = 'Outdoor')
    """

    TOTAL_FEED_SUPPLY = """
        SELECT SUM(quantity) AS "Общее количество корма"
        FROM "Feed_supply"
    """

    # JP6: GROUP BY с HAVING
    AVIARY_AVG_SQUARE = """
        SELECT type AS "Тип вольера", ROUND(AVG(square),
2) AS "Средняя площадь"
        FROM "Aviary"
        GROUP BY type
        HAVING AVG(square) > 20
    """

    # JP6: INTERSECT, EXCEPT
    ANIMALS_BOTH_CHEAP_AND_EXPENSIVE = """
        SELECT a.id, a.breed, a.type
        FROM "Animal" a
        JOIN "Animal_medical_procedure" amp ON a.id =
amp.id_animal
        JOIN "Medical_procedure" mp ON amp.id_medical =
mp.id
        WHERE mp.cost < 2000

        INTERSECT

        SELECT a.id, a.breed, a.type
        FROM "Animal" a
        JOIN "Animal_medical_procedure" amp ON a.id =
amp.id_animal
        JOIN "Medical_procedure" mp ON amp.id_medical =
mp.id
        WHERE mp.cost > 2000
        ORDER BY id
    """

    EMPTY_AVIARIES = """

```

```

SELECT id, type, status, location
FROM "Aviary"

EXCEPT

SELECT av.id, av.type, av.status, av.location
FROM "Aviary" av
JOIN "Animal" a ON av.id = a.id_aviary
"""

# Дополнительные запросы
ANIMALS_WITH_DETAILS = """
SELECT      a.id,          a.breed,          a.type,
a.state_of_health,
            av.location      AS      aviary_location,
av.status AS aviary_status,
            e.snp    AS    employee_name,    e.post    AS
employee_post
FROM "Animal" a
JOIN "Aviary" av ON a.id_aviary = av.id
JOIN "Employee" e ON a.id_employee = e.id
ORDER BY a.id
"""

VOLUNTEER_WORKLOAD = """
SELECT      v.snp    AS    volunteer_name,    v.duty,
COUNT(av.id_animal) AS animals_count
FROM "Volunteer" v
LEFT JOIN "Animal_volunteer" av ON v.id =
av.id_volunteer
GROUP BY v.id, v.snp, v.duty
ORDER BY animals_count DESC
"""

MEDICAL_COSTS_BY_ANIMAL = """
SELECT a.breed, a.type, COUNT(amp.id_medical)
AS procedures_count,
      SUM(mp.cost) AS total_cost
FROM "Animal" a
JOIN "Animal_medical_procedure" amp ON a.id =
amp.id_animal
JOIN "Medical_procedure" mp ON amp.id_medical =
mp.id
GROUP BY a.id, a.breed, a.type
ORDER BY total_cost DESC
"""

# Словарь для доступа к запросам
QUERIES = {
    "animals_treatment": ANIMALS_TREATMENT,
    "employees_recent": EMPLOYEES_RECENT,

```

```

        "animals_active_aviaries":
ANIMALS_ACTIVE_AVIARIES,
        "volunteers_search": VOLUNTEERS_SEARCH,
        "medical_procedures_cost_range":
MEDICAL_PROCEDURES_COST_RANGE,
        "animals_with_volunteers":
ANIMALS_WITH_VOLUNTEERS,
        "animals_count_by_type": ANIMALS_COUNT_BY_TYPE,
        "unique_breeds_active_aviaries":
UNIQUE_BREEDS_IN_ACTIVE_AVIARIES,
        "max_outdoor_square": MAX_OUTDOOR_SQUARE,
        "total_feed_supply": TOTAL_FEED_SUPPLY,
        "aviary_avg_square": AVIARY_AVG_SQUARE,
        "animals_both_cheap_expensive":
ANIMALS_BOTH_CHEAP_AND_EXPENSIVE,
        "empty_aviaries": EMPTY_AVIARIES,
        "animals_with_details": ANIMALS_WITH_DETAILS,
        "volunteer_workload": VOLUNTEER_WORKLOAD,
        "medical_costs_by_animal":
MEDICAL_COSTS_BY_ANIMAL
    }

```

```

# Описания запросов
QUERY_DESCRIPTIONS = {
    "animals_treatment": "Животные на лечении или
карантине",
    "employees_recent": "Сотрудники, принятые после
2021 года",
    "animals_active_aviaries": "Животные в активных
вольерах",
    "volunteers_search": "Волонтеры по уходу за
собаками/кошками",
    "medical_procedures_cost_range": "Медицинские
процедуры (2000-3500 руб)",
    "animals_with_volunteers": "Животные с
закрепленными волонтерами",
    "animals_count_by_type": "Статистика животных
по типам",
    "unique_breeds_active_aviaries": "Уникальные
породы в активных вольерах",
    "max_outdoor_square": "Вольеры Outdoor с
максимальной площадью",
    "total_feed_supply": "Общее количество
поставленного корма",
    "aviary_avg_square": "Типы вольеров со средней
площадью > 20 м²",
    "animals_both_cheap_expensive": "Животные с
дешевыми и дорогими процедурами",
    "empty_aviaries": "Пустые вольеры",
    "animals_with_details": "Животные с детальной
информацией",
    "volunteer_workload": "Нагрузка на волонтеров",

```

```

        "medical_costs_by_animal": "Медицинские расходы
по животным"
    }

import psycopg2
from psycopg2.extras import RealDictCursor
import os
from typing import List, Dict, Any, Optional

# Параметры подключения к БД (измените под свои)
DB_PARAMS = {
    'dbname': 'animal_shelter',
    'user': 'postgres',
    'password': 'atymelancholy', # Ваш пароль
    'host': 'localhost',
    'port': '5432'
}

class DatabaseManager:
    """Класс для управления подключением и запросами к
БД"""

    def __init__(self):
        self.connection = None

    def get_connection(self):
        """Устанавливает соединение с базой данных"""
        if not self.connection or self.connection.closed:
            self.connection = psycopg2.connect(**DB_PARAMS)
        return self.connection

    def close_connection(self):
        """Закрывает соединение с БД"""
        if self.connection and not self.connection.closed:
            self.connection.close()
            print("Подключение к базе данных закрыто")

    def execute_query(self, sql: str, params: Optional[List] = None, fetch: bool = True) -> List[Dict[str, Any]]:
        """
        Выполняет SQL-запрос и возвращает результат в
        виде списка словарей
        """
        conn = self.get_connection()
        cursor = conn.cursor(cursor_factory=RealDictCursor)

```



```

        try:
            cursor.execute(sql, params)

            if fetch:
                if
sql.strip().upper().startswith('SELECT'):
                    result = cursor.fetchall()
                else:
                    conn.commit()
                    result = [{"message": "Query
executed successfully", "rows_affected": cursor.rowcount}]
                else:
                    conn.commit()
                    result = [{"message": "Query executed
successfully", "rows_affected": cursor.rowcount}]

            return [dict(row) for row in result] if
result else []

        except Exception as e:
            conn.rollback()
            raise e
        finally:
            cursor.close()

    def get_table_data(self, table_name: str, filters:
Optional[Dict] = None,
                                limit: int = 100, offset: int =
0) -> List[Dict[str, Any]]:
        """
        Получает данные из таблицы с возможностью
фильтрации и пагинации
        """
        query = f'SELECT * FROM "{table_name}"'
        params = []

        if filters:
            where_clauses = []
            for key, value in filters.items():
                if value is not None:
                    where_clauses.append(f'"{key}" =
%s')
                    params.append(value)

            if where_clauses:
                query += " WHERE " + " AND "
                query = query.join(where_clauses)

            query += f" LIMIT {limit} OFFSET {offset}"

        return self.execute_query(query, params)

```

```

        def insert_record(self, table_name: str, data:
Dict[str, Any]) -> List[Dict[str, Any]]:
        """
        Добавляет новую запись в таблицу
        """
        # Фильтруем поля, которые могут быть ID (они
генерируются автоматически)
        filtered_data = {k: v for k, v in data.items()
if k not in ['id']}

        columns = ', '.join([f'"{col}"' for col in
filtered_data.keys()])
        placeholders = ', '.join(['%s'] *
len(filtered_data))
        values = list(filtered_data.values())

        query = f'INSERT INTO "{table_name}" ({columns})
VALUES ({placeholders}) RETURNING *'

        return self.execute_query(query, values)

        def update_record(self, table_name: str, record_id:
int, data: Dict[str, Any]) -> List[Dict[str, Any]]:
        """
        Обновляет запись в таблице по ID
        """
        # Убираем ID из обновляемых полей
        filtered_data = {k: v for k, v in data.items()
if k not in ['id']}

        if not filtered_data:
            return [{"message": "No fields to update"}]

        set_clause = ', '.join([f'"{key}" = %s' for key
in filtered_data.keys()])
        values = list(filtered_data.values())
        values.append(record_id)

        query = f'UPDATE "{table_name}" SET {set_clause}
WHERE id = %s RETURNING *'

        return self.execute_query(query, values)

        def delete_record(self, table_name: str, record_id:
int) -> List[Dict[str, Any]]:
        """
        Удаляет запись из таблицы по ID
        """
        query = f'DELETE FROM "{table_name}" WHERE id =
%s'

        return self.execute_query(query, [record_id],
fetch=False)

```

```

def get_table_names(self) -> List[str]:
    """
    Получает список всех таблиц в схеме public
    """
    query = """
        SELECT table_name
        FROM information_schema.tables
        WHERE table_schema = 'public' AND table_type
= 'BASE TABLE'
        ORDER BY table_name
    """
    result = self.execute_query(query)
    return [row['table_name'] for row in result]

# Создаем глобальный экземпляр менеджера БД
db_manager = DatabaseManager()

import os
from datetime import datetime
from typing import List, Optional
from database import db_manager

class BackupManager:
    """Класс для создания резервных копий"""

    def __init__(self, backup_dir: str = "exports"):
        self.backup_dir = backup_dir
        os.makedirs(self.backup_dir, exist_ok=True)

    def create_backup(self, table_name: Optional[str] =
None) -> str:
        """
        Создает резервную копию таблицы или всей базы
данных

        Возвращает путь к файлу бэкапа
        """
        timestamp =
datetime.now().strftime("%Y%m%d_%H%M%S")

        if table_name:
            filename =
f"backup_{table_name}_{timestamp}.sql"
            backup_content =
self._backup_table(table_name)
        else:
            filename =
f"backup_full_database_{timestamp}.sql"
            backup_content = self._backup_all_tables()

```

```

        filepath = os.path.join(self.backup_dir,
filename)

        with open(filepath, 'w', encoding='utf-8') as f:
            f.write(backup_content)

        return filepath

    def _backup_table(self, table_name: str) -> str:
        """Создает SQL дамп одной таблицы"""
        content = []
        content.append(f"--
=====")
        content.append(f"--      Backup      of      table:
{table_name}")
        content.append(f"--      Created      at:
{datetime.now()}")
        content.append(f"--
=====\\n")

        # Получаем структуру таблицы
        structure_query = f"""
        SELECT column_name, data_type, is_nullable
        FROM information_schema.columns
        WHERE table_name = '{table_name}'
        ORDER BY ordinal_position
        """

        # Получаем данные из таблицы
        data = db_manager.get_table_data(table_name,
limit=10000)

        if data:
            columns = list(data[0].keys())
            content.append(f"INSERT INTO
\\'{table_name}\\' ({', '.join([f'\\'{c}\\'' for c in columns])})
VALUES")

            rows = []
            for row in data:
                values = []
                for col in columns:
                    val = row[col]
                    if val is None:
                        values.append("NULL")
                    elif isinstance(val, (int, float)):
                        values.append(str(val))
                    else:
                        # Экранируем кавычки
                        escaped_val =
str(val).replace("'", "'")

```

```

values.append(f"'{escaped_val}'")
        rows.append(f"({'', ' '.join(values)})")

        content.append(",\n".join(rows) + ";")

    return "\n".join(content)

    def _backup_all_tables(self) -> str:
        """Создает SQL дампы всех таблиц"""
        content = []
        content.append(f"--
=====")
        content.append(f"-- FULL DATABASE BACKUP")
        content.append(f"--          Created          at:
{datetime.now()}")
        content.append(f"--
=====\\n")

        tables = db_manager.get_table_names()

        for table in tables:
            content.append(self._backup_table(table))
            content.append("\\n")

        return "\n".join(content)

    def restore_from_backup(self, filepath: str) ->
bool:
        """
        Восстанавливает данные из файла бэкапа
        (Упрощенная версия - выполняет SQL команды из
        файла)
        """
        try:
            with open(filepath, 'r', encoding='utf-8')
as f:
                sql_content = f.read()

                # Разделяем на отдельные команды и выполняем
                # (В реальном проекте нужен более надежный
                парсер)

                commands = sql_content.split(';')
                for cmd in commands:
                    cmd = cmd.strip()
                    if cmd and not cmd.startswith('--'):
                        try:
                            db_manager.execute_query(cmd,
fetch=False)
                        except Exception as e:
                            print(f"Ошибка при выполнении:
{cmd[:100]}... - {e}")

```

```
        return True
    except Exception as e:
        print(f"Ошибка восстановления: {e}")
        return False
```

```
backup_manager = BackupManager()
```